

◆ ДОМЕН 4: C# / .NET — ПОЛНАЯ ДЕТАЛИЗАЦИЯ (ОТ НУЛЯ ДО ЭКСПЕРТА)

СТРУКТУРА КАЖДОЙ ТЕМЫ

Для каждой темы я даю:

- **Что нужно знать** — теория для понимания
 - **Вопросы на собеседовании** (Junior → Senior → Staff)
 - **Практика** — конкретные задания для закрепления
-

РАЗДЕЛ 4.1: БАЗОВЫЙ СИНТАКСИС И ООП (JUNIOR LEVEL)

4.1.1. Основы языка (Junior Level)

№ 4.1.1.1 — Типы данных

- **Что нужно знать:** Value types (int, float, double, bool, char, struct, enum) vs Reference types (class, interface, delegate, string, array)
- **Вопросы на собеседовании:** "В чем разница между value type и reference type?"
- **Практика:** Написать программу, показывающую, что int хранится в стеке, а object в куче

№ 4.1.1.2 — Коробочная/распаковка (Boxing/Unboxing)

- **Что нужно знать:** преобразование value type → object, потери производительности
- **Вопросы на собеседовании:** "Что такое boxing и unboxing? Почему это плохо?"
- **Практика:** Написать бенчмарк, показывающий разницу между int и object при работе с коллекциями

№ 4.1.1.3 — Переменные

- **Что нужно знать:** объявление, инициализация, var (неявная типизация), область видимости
- **Вопросы на собеседовании:** "Когда использовать var?"
- **Практика:** Написать программу с var для сложных типов и сравнить с явным объявлением

№ 4.1.1.4 — Условные операторы

- **Что нужно знать:** if, else, switch, switch с pattern matching (C# 7+)
- **Вопросы на собеседовании:** "Что такое pattern matching в C#?"
- **Практика:** Написать программу, использующую switch с when для разных типов

№ 4.1.1.5 — Циклы

- **Что нужно знать:** for, foreach, while, do-while
- **Вопросы на собеседовании:** "В чем разница между for и foreach?"
- **Практика:** Написать программу, проходящую по списку и изменяющую элементы

№ 4.1.1.6 — Массивы

- **Что нужно знать:** одномерные, многомерные (rectangular), зубчатые (jagged)
- **Вопросы на собеседовании:** "В чем разница между прямоугольным и зубчатым массивом?"
- **Практика:** Написать программу, работающую с матрицами и jagged arrays

№ 4.1.1.7 — Строки

- **Что нужно знать:** string (immutable), StringBuilder (mutable), интернирование
- **Вопросы на собеседовании:** "Почему строки immutable? Когда использовать StringBuilder?"
- **Практика:** Написать бенчмарк конкатенации 10000 строк через + и StringBuilder

№ 4.1.1.8 — Перечисления (enum)

- **Что нужно знать:** базовый тип (int по умолчанию), флаги ([Flags])
- **Вопросы на собеседовании:** "Как использовать [Flags] для битовых масок?"
- **Практика:** Создать enum FileAccess с Read, Write, ReadWrite

№ 4.1.1.9 — Классы

- **Что нужно знать:** поля, свойства (get/set, auto-properties), методы, статические члены

- **Вопросы на собеседовании:** "В чем разница между полем и свойством?"
- **Практика:** Создать класс Person с авто-свойствами и вычисляемым свойством FullName

№ 4.1.1.10 — Конструкторы

- **Что нужно знать:** по умолчанию, с параметрами, статические, цепочка вызовов (this())
- **Вопросы на собеседовании:** "Что такое статический конструктор и когда он вызывается?"
- **Практика:** Создать класс с несколькими конструкторами, использующими this()

№ 4.1.1.11 — Структуры (struct)

- **Что нужно знать:** value type, отличия от класса (нет наследования, но есть интерфейсы)
- **Вопросы на собеседовании:** "Когда использовать struct вместо class?"
- **Практика:** Создать структуру Point с полями X, Y и методом DistanceTo

№ 4.1.1.12 — Интерфейсы

- **Что нужно знать:** явная и неявная реализация, множественное наследование интерфейсов
- **Вопросы на собеседовании:** "В чем разница между интерфейсом и абстрактным классом?"
- **Практика:** Создать интерфейс IDrawable и реализовать его в нескольких классах

№ 4.1.1.13 — Наследование

- **Что нужно знать:** sealed (запрет наследования), abstract (абстрактный класс)
- **Вопросы на собеседовании:** "Что такое абстрактный класс? Можно ли создать его экземпляр?"
- **Практика:** Создать иерархию Animal → Dog, Cat с виртуальными методами

№ 4.1.1.14 — Полиморфизм

- **Что нужно знать:** виртуальные методы (virtual/override), скрытие (new), base
- **Вопросы на собеседовании:** "В чем разница между override и new в C#?"
- **Практика:** Создать иерархию с virtual и override, показать поведение при приведении к базовому типу

№ 4.1.1.15 — Исключения

- **Что нужно знать:** try/catch/finally/throw, создание своих исключений
- **Вопросы на собеседовании:** "Какие типы исключений есть в C#?"
- **Практика:** Создать собственное исключение InvalidAgeException

№ 4.1.1.16 — Делегаты

- **Что нужно знать:** Action, Func, Predicate, создание своих делегатов
- **Вопросы на собеседовании:** "Что такое делегат? В чем разница между Action и Func?"
- **Практика:** Написать программу с делегатом для обработки событий

№ 4.1.1.17 — События (events)

- **Что нужно знать:** event модификатор, подписка/отписка (+=, -=)
- **Вопросы на собеседовании:** "В чем разница между делегатом и событием?"
- **Практика:** Создать класс Button с событием Click и подписаться на него

4.1.2. Коллекции и LINQ (Junior → Middle Level)

№ 4.1.2.1 — List<T>

- **Что нужно знать:** внутреннее устройство (массив с амортизацией), производительность
- **Вопросы на собеседовании:** "Как работает List<T> под капотом?"
- **Практика:** Написать программу, измеряющую время вставки в List<T>

№ 4.1.2.2 — Dictionary<TKey, TValue>

- **Что нужно знать:** внутреннее устройство (хеш-таблица), коллизии

- **Вопросы на собеседовании:** "Как работает Dictionary в C#? Что такое load factor?"
- **Практика:** Написать программу с Dictionary, показать производительность поиска

№ 4.1.2.3 — HashSet<T>

- **Что нужно знать:** хранит уникальные элементы, внутреннее устройство (хеш-таблица без значений)
- **Вопросы на собеседовании:** "Когда использовать HashSet вместо List?"
- **Практика:** Написать программу, удаляющую дубликаты с HashSet

№ 4.1.2.4 — Queue<T>, Stack<T>

- **Что нужно знать:** FIFO, LIFO
- **Вопросы на собеседовании:** "Чем отличаются Queue и Stack?"
- **Практика:** Реализовать producer-consumer с Queue<T>

№ 4.1.2.5 — LinkedList<T>

- **Что нужно знать:** двусвязный список, быстрая вставка/удаление
- **Вопросы на собеседовании:** "Когда использовать LinkedList вместо List?"
- **Практика:** Сравнить производительность List и LinkedList при вставке в начало

№ 4.1.2.6 — LINQ

- **Что нужно знать:** query syntax (from ... select) vs method syntax (.Select().Where())
- **Вопросы на собеседовании:** "Что такое LINQ? В чем разница между двух синтаксисов?"
- **Практика:** Написать LINQ-запрос для группировки заказов по клиентам

№ 4.1.2.7 — LINQ (отложенное выполнение)

- **Что нужно знать:** IEnumerable<T> vs IQueryable<T>
- **Вопросы на собеседовании:** "В чем разница между IEnumerable и IQueryable?"

- **Практика:** Написать программу, показывающую отложенное выполнение IEnumerable

№ 4.1.2.8 — LINQ (операторы)

- **Что нужно знать:** Select, Where, GroupBy, Join, Aggregate, OrderBy
- **Вопросы на собеседовании:** "Как объединить две коллекции в LINQ?"
- **Практика:** Написать запрос, объединяющий список пользователей и заказов

№ 4.1.2.9 — LINQ (оконные функции)

- **Что нужно знать:** Take, Skip, Distinct, Union, Intersect, Except
- **Вопросы на собеседовании:** "Как получить первые 10 элементов с пропуском первых 5?"
- **Практика:** Написать запрос для пагинации с Skip и Take

4.1.3. Практические задания (Junior Level)

№ 4.1.3.1

- **Задание:** Реализовать класс Stack<T> с использованием массива (с амортизацией)
- **Что проверяет:** Понимание работы со стеком и памятью

№ 4.1.3.2

- **Задание:** Реализовать программу для работы с коллекцией студентов (CRUD + LINQ для поиска/группировки)
- **Что проверяет:** Знание коллекций и LINQ

№ 4.1.3.3

- **Задание:** Реализовать класс BankAccount с событиями при пополнении/списании
- **Что проверяет:** Понимание событий и делегатов

№ 4.1.3.4

- **Задание:** Написать программу для парсинга CSV файла и вывода статистики с LINQ
 - **Что проверяет:** Практическое применение LINQ
-

РАЗДЕЛ 4.2: АСИНХРОННОСТЬ И МНОГОПОТОЧНОСТЬ (MIDDLE → SENIOR)

4.2.1. Асинхронность (Middle Level)

№ 4.2.1.1 — async/await

- **Что нужно знать:** как работает state machine, возвращаемые типы (Task, Task<T>, void)
- **Вопросы на собеседовании:** "Что происходит под капотом при await?"
- **Практика:** Написать асинхронный метод для загрузки данных с API

№ 4.2.1.2 — State Machine

- **Что нужно знать:** как компилятор преобразует async метод в класс с состояниями
- **Вопросы на собеседовании:** "Как реализована асинхронность в C# на уровне IL?"
- **Практика:** Дизассемблировать асинхронный метод в ILSpy и показать state machine

№ 4.2.1.3 — ValueTask<T>

- **Что нужно знать:** когда использовать вместо Task<T> (для часто кэшируемых результатов)
- **Вопросы на собеседовании:** "В чем разница между Task и ValueTask?"
- **Практика:** Написать асинхронный метод, возвращающий кэшированный результат через ValueTask

№ 4.2.1.4 — CancellationToken

- **Что нужно знать:** отмена асинхронных операций, CancelAfter()
- **Вопросы на собеседовании:** "Как отменить асинхронную операцию?"
- **Практика:** Написать метод с CancellationToken и отменить его через 1 секунду

№ 4.2.1.5 — Task.WhenAll / WhenAny

- **Что нужно знать:** параллельный запуск задач, обработка результатов
- **Вопросы на собеседовании:** "Как дождаться завершения нескольких асинхронных задач?"
- **Практика:** Загрузить данные с 10 URL параллельно с WhenAll

№ 4.2.1.6 — IEnumerable<T> (C# 8)

- **Что нужно знать:** асинхронные стримы данных
- **Вопросы на собеседовании:** "Когда использовать IEnumerable?"
- **Практика:** Написать метод, возвращающий IEnumerable и отдающий данные по мере получения

№ 4.2.1.7 — ConfigureAwait(false)

- **Что нужно знать:** когда и зачем использовать, предотвращение deadlock
- **Вопросы на собеседовании:** "Почему важно использовать ConfigureAwait(false) в библиотеках?"
- **Практика:** Написать библиотеку с ConfigureAwait(false) и показать разницу в UI-приложении

№ 4.2.1.8 — Потокобезопасность в асинхронном коде

- **Что нужно знать:** Interlocked, Concurrent коллекции
- **Вопросы на собеседовании:** "Как безопасно обновлять счетчик в асинхронном коде?"
- **Практика:** Использовать Interlocked.Increment в параллельных асинхронных задачах

4.2.2. Многопоточность (Middle → Senior Level)

№ 4.2.2.1 — Thread

- **Что нужно знать:** создание, Start, Join, IsBackground
- **Вопросы на собеседовании:** "Когда использовать Thread вместо Task?"
- **Практика:** Создать поток и запустить его

№ 4.2.2.2 — ThreadPool

- **Что нужно знать:** пул потоков для выполнения задач
- **Вопросы на собеседовании:** "Зачем нужен ThreadPool?"
- **Практика:** Запустить несколько задач через ThreadPool.QueueUserWorkItem

№ 4.2.2.3 — Task vs Thread

- **Что нужно знать:** Task — абстракция над ThreadPool, более легковесный
- **Вопросы на собеседовании:** "В чем разница между Task и Thread?"
- **Практика:** Сравнить количество памяти для Task и Thread

№ 4.2.2.4 — lock (Monitor)

- **Что нужно знать:** синхронизация доступа, Monitor.Enter/Exit
- **Вопросы на собеседовании:** "Как работает lock в C#? Что под капотом?"
- **Практика:** Написать программу с lock для защиты общего ресурса

№ 4.2.2.5 — Mutex, Semaphore, EventWaitHandle

- **Что нужно знать:** межпроцессная синхронизация
- **Вопросы на собеседовании:** "В чем разница между Mutex и Semaphore?"
- **Практика:** Написать программу, синхронизирующую два процесса через Mutex

№ 4.2.2.6 — ReaderWriterLockSlim

- **Что нужно знать:** read-write lock для чтения/записи
- **Вопросы на собеседовании:** "Когда использовать ReaderWriterLockSlim?"
- **Практика:** Реализовать кеш с ReaderWriterLockSlim

№ 4.2.2.7 — Concurrent коллекции

- **Что нужно знать:** ConcurrentBag<T>, ConcurrentDictionary<TKey,TValue>, ConcurrentQueue<T>
- **Вопросы на собеседовании:** "В чем преимущество ConcurrentDictionary перед Dictionary с lock?"
- **Практика:** Написать производителя/потребителя с ConcurrentQueue

№ 4.2.2.8 — Interlocked

- **Что нужно знать:** атомарные операции (Increment, Decrement, Exchange, CompareExchange)
- **Вопросы на собеседовании:** "Как инкрементировать счетчик без lock?"

- **Практика:** Использовать Interlocked.Increment в многопоточном коде

№ 4.2.2.9 — SpinLock

- **Что нужно знать:** спинлок для коротких критических секций
- **Вопросы на собеседовании:** "Когда использовать SpinLock вместо lock?"
- **Практика:** Написать бенчмарк сравнения lock и SpinLock

№ 4.2.2.10 — ThreadLocal<T>

- **Что нужно знать:** локальные данные для каждого потока
- **Вопросы на собеседовании:** "Как создать переменную, уникальную для каждого потока?"
- **Практика:** Использовать ThreadLocal<Random> для генерации случайных чисел в потоках

№ 4.2.2.11 — Barrier

- **Что нужно знать:** синхронизация группы потоков (все дожидаются друг друга)
- **Вопросы на собеседовании:** "Что такое Barrier и когда его использовать?"
- **Практика:** Написать программу с 5 потоками, достигающими барьера одновременно

№ 4.2.2.12 — SemaphoreSlim

- **Что нужно знать:** легковесный семафор для асинхронного кода
- **Вопросы на собеседовании:** "В чем разница между Semaphore и SemaphoreSlim?"
- **Практика:** Ограничить количество параллельных задач до 10 с SemaphoreSlim

4.2.3. Практические задания (Middle → Senior Level)

№ 4.2.3.1

- **Задание:** Реализовать асинхронный сканер веб-страниц (загружает URL параллельно)
- **Что проверяет:** Знание async/await и WhenAll

№ 4.2.3.2

- **Задание:** Реализовать producer-consumer с BlockingCollection<T>
- **Что проверяет:** Знание многопоточности

№ 4.2.3.3

- **Задание:** Реализовать thread pool с очередью задач (свой пул)
- **Что проверяет:** Глубокое понимание многопоточности

№ 4.2.3.4

- **Задание:** Написать бенчмарк для сравнения lock, Interlocked и ConcurrentDictionary
- **Что проверяет:** Понимание производительности синхронизации

РАЗДЕЛ 4.3: CLR И УПРАВЛЕНИЕ ПАМЯТЬЮ (SENIOR → STAFF)

4.3.1. Сборщик мусора (GC) — Senior Level

№ 4.3.1.1 — GC (Garbage Collector)

- **Что нужно знать:** как работает, поколения (0, 1, 2), LOH, эвакуация объектов
- **Вопросы на собеседовании:** "Как работает сборщик мусора в .NET?"
- **Практика:** Написать программу, создающую 100000 объектов и снять дамп GC

№ 4.3.1.2 — Поколения

- **Что нужно знать:** Gen0 (новые объекты), Gen1 (выжившие после Gen0), Gen2 (долгоживущие)
- **Вопросы на собеседовании:** "Почему GC делит объекты на поколения?"
- **Практика:** Проанализировать работу GC через dotnet-gcdump

№ 4.3.1.3 — LOH (Large Object Heap)

- **Что нужно знать:** объекты > 85000 байт, не собирается часто
- **Вопросы на собеседовании:** "Что такое LOH и почему он особенный?"
- **Практика:** Написать программу, создающую большие массивы и показать, как они попадают в LOH

№ 4.3.1.4 — POH (Pinned Object Heap)

- **Что нужно знать:** закрепленные объекты (Pinned)
- **Вопросы на собеседовании:** "Что такое POH и зачем он нужен?"
- **Практика:** Создать pinned объект через GCHandle.Alloc

№ 4.3.1.5 — Типы GC

- **Что нужно знать:** Workstation (однопоточный), Server (многопоточный)
- **Вопросы на собеседовании:** "В чем разница между Workstation и Server GC?"
- **Практика:** Настроить Server GC в .csproj и показать разницу в производительности

№ 4.3.1.6 — IDisposable и using

- **Что нужно знать:** освобождение неуправляемых ресурсов
- **Вопросы на собеседовании:** "Как реализовать IDisposable правильно?"
- **Практика:** Реализовать Dispose Pattern для класса с неуправляемым ресурсом

№ 4.3.1.7 — Финализаторы (~ClassName)

- **Что нужно знать:** вызываются перед GC, GC.SuppressFinalize
- **Вопросы на собеседовании:** "В чем разница между Dispose и Finalize?"
- **Практика:** Реализовать класс с финализатором и показать, когда он вызывается

№ 4.3.1.8 — GC.Collect()

- **Что нужно знать:** принудительная сборка, когда использовать (почти никогда)
- **Вопросы на собеседовании:** "Почему не рекомендуется вызывать GC.Collect()?"
- **Практика:** Написать программу с GC.Collect и показать влияние на производительность

№ 4.3.1.9 — WeakReference

- **Что нужно знать:** предотвращение утечек памяти через события

- **Вопросы на собеседовании:** "Как избежать утечки памяти при подписке на события?"
- **Практика:** Использовать WeakReference для подписки на событие

№ 4.3.1.10 — GCHandle

- **Что нужно знать:** закрепка объектов для P/Invoke, типы (Normal, Pinned, Weak)
 - **Вопросы на собеседовании:** "Что такое GCHandleType.Pinned и зачем он нужен?"
 - **Практика:** Закрепить массив для передачи в нативный код через P/Invoke
-

4.3.2. JIT и внутренности CLR (Staff Level)

№ 4.3.2.1 — JIT (Just-In-Time Compiler)

- **Что нужно знать:** как компилирует IL в машинный код, RyuJIT
- **Вопросы на собеседовании:** "Как работает JIT-компиляция?"
- **Практика:** Проанализировать JIT-компиляцию через дизассемблер

№ 4.3.2.2 — Tiered Compilation

- **Что нужно знать:** сначала быстрая JIT, потом оптимизированная
- **Вопросы на собеседовании:** "Что такое Tiered Compilation и зачем она нужна?"
- **Практика:** Включить Tiered Compilation в .csproj и показать разницу в производительности

№ 4.3.2.3 — Native AOT

- **Что нужно знать:** компиляция в нативный код (без JIT), уменьшение времени запуска
- **Вопросы на собеседовании:** "В чем преимущество Native AOT перед JIT?"
- **Практика:** Скомпилировать приложение с Native AOT (`<PublishAot>true</PublishAot>`)

№ 4.3.2.4 — IL-код

- **Что нужно знать:** чтение IL через ILSpy/dnSpy, понимание инструкций (ldloc, stloc, call, newobj)

- **Вопросы на собеседовании:** "Как выглядит IL-код для простого метода?"
- **Практика:** Дизассемблировать метод в ILSpy и объяснить инструкции

№ 4.3.2.5 — P/Invoke

- **Что нужно знать:** вызов нативного кода из C#, маршаллинг (Marshal класс)
- **Вопросы на собеседовании:** "Как вызвать функцию из C++ в C#?"
- **Практика:** Написать P/Invoke для функции MessageBox из user32.dll

№ 4.3.2.6 — Атрибуты

- **Что нужно знать:** создание своих атрибутов, Conditional, Obsolete
- **Вопросы на собеседовании:** "Как создать свой атрибут и использовать его?"
- **Практика:** Создать атрибут [Author("John")] для документирования методов

№ 4.3.2.7 — Рефлексия (Reflection)

- **Что нужно знать:** получение типов, создание объектов, вызов методов во время выполнения
- **Вопросы на собеседовании:** "Когда использовать рефлексия? Какие есть минусы?"
- **Практика:** Написать программу, использующую рефлексия для вызова метода по имени

№ 4.3.2.8 — System.Runtime.CompilerServices

- **Что нужно знать:** Unsafe класс (низкоуровневая работа с памятью)
- **Вопросы на собеседовании:** "Как работать с памятью на низком уровне в C#?"
- **Практика:** Написать программу с Unsafe для работы с указателями

№ 4.3.2.9 — Span<T> и Memory<T>

- **Что нужно знать:** безопасная работа с памятью (без копирования)
- **Вопросы на собеседовании:** "Что такое Span<T> и в чем его преимущество?"
- **Практика:** Переписать парсер с Span<T> для уменьшения аллокаций

№ 4.3.2.10 — ref struct

- **Что нужно знать:** структуры, которые могут быть только в стеке
- **Вопросы на собеседовании:** "Что такое ref struct? Когда ее использовать?"
- **Практика:** Создать ref struct для безопасной работы с памятью

№ 4.3.2.11 — System.Text.Json

- **Что нужно знать:** высокопроизводительный JSON-сериализатор (замена Newtonsoft.Json)
- **Вопросы на собеседовании:** "В чем преимущество System.Text.Json перед Newtonsoft.Json?"
- **Практика:** Написать сериализатор/десериализатор с System.Text.Json

№ 4.3.2.12 — MemoryCache

- **Что нужно знать:** кеширование в памяти, политики вытеснения
- **Вопросы на собеседовании:** "Как реализовать кеш в C#?"
- **Практика:** Реализовать кеш с MemoryCache и TTL

4.3.3. Практические задания (Senior → Staff Level)

№ 4.3.3.1

- **Задание:** Реализовать Dispose Pattern для класса с управляемыми и неуправляемыми ресурсами
- **Что проверяет:** Понимание управления памятью

№ 4.3.3.2

- **Задание:** Снять dotnet-gcdump для приложения, проанализировать поколения и LOH
- **Что проверяет:** Знание профилирования памяти

№ 4.3.3.3

- **Задание:** Реализовать свой Span<T>-подобный тип (безопасный доступ к массиву)
- **Что проверяет:** Понимание работы с памятью

№ 4.3.3.4

- **Задание:** Написать программу с P/Invoke для вызова нативной библиотеки
 - **Что проверяет:** Понимание взаимодействия с нативным кодом
-

РАЗДЕЛ 4.4: [ASP.NET](#) CORE И ВЕБ-РАЗРАБОТКА (MIDDLE → SENIOR)

4.4.1. [ASP.NET](#) Core (Middle Level)

№ 4.4.1.1 — WebApplication

- **Что нужно знать:** создание, настройка, хостинг
- **Вопросы на собеседовании:** "Как создать минимальное веб-приложение на [ASP.NET](#) Core?"
- **Практика:** Создать API с одним эндпоинтом /hello

№ 4.4.1.2 — Minimal API

- **Что нужно знать:** app.MapGet(), app.MapPost(), маршрутизация
- **Вопросы на собеседовании:** "В чем отличие Minimal API от Controller-based?"
- **Практика:** Написать CRUD API для Todo с Minimal API

№ 4.4.1.3 — Controller-based API

- **Что нужно знать:** [ApiController], [Route], [HttpGet], [HttpPost]
- **Вопросы на собеседовании:** "Как создать контроллер в [ASP.NET](#) Core?"
- **Практика:** Создать контроллер UsersController с методами CRUD

№ 4.4.1.4 — Dependency Injection (DI)

- **Что нужно знать:** AddScoped, AddTransient, AddSingleton
- **Вопросы на собеседовании:** "В чем разница между Scoped, Transient и Singleton?"
- **Практика:** Создать сервис IEmailService и зарегистрировать в DI

№ 4.4.1.5 — Middleware

- **Что нужно знать:** обработка запросов, порядок выполнения
- **Вопросы на собеседовании:** "Как работает Middleware в [ASP.NET](#) Core?"

- **Практика:** Написать middleware для логирования запросов

№ 4.4.1.6 — Конфигурация

- **Что нужно знать:** appsettings.json, переменные окружения, IConfiguration
- **Вопросы на собеседовании:** "Как читать конфигурацию в [ASP.NET](#) Core?"
- **Практика:** Настроить чтение строки подключения из appsettings.json

№ 4.4.1.7 — Логирование

- **Что нужно знать:** ILogger<T>, Serilog, уровни логирования
- **Вопросы на собеседовании:** "Как настроить логирование в [ASP.NET](#) Core?"
- **Практика:** Настроить Serilog с записью в файл и консоль

№ 4.4.1.8 — Аутентификация

- **Что нужно знать:** JWT (Bearer), [Authorize], ClaimsPrincipal
- **Вопросы на собеседовании:** "Как реализовать аутентификацию по JWT?"
- **Практика:** Настроить JWT-аутентификацию для API

№ 4.4.1.9 — gRPC

- **Что нужно знать:** Protocol Buffers, сервер и клиент
- **Вопросы на собеседовании:** "Когда использовать gRPC вместо REST?"
- **Практика:** Создать gRPC-сервис на C# и клиент на C++

4.4.2. [ASP.NET](#) Core (Продвинутый) — Senior Level

№ 4.4.2.1 — Health Checks

- **Что нужно знать:** AddHealthChecks(), UseHealthChecks()
- **Вопросы на собеседовании:** "Как реализовать проверку готовности сервиса?"
- **Практика:** Добавить Health Checks для проверки БД и внешних сервисов

№ 4.4.2.2 — Polly

- **Что нужно знать:** Retry, Circuit Breaker, Timeout, Bulkhead
- **Вопросы на собеседовании:** "Как добавить повторные попытки для HTTP-запросов?"
- **Практика:** Настроить Polly с Retry для HttpClient

№ 4.4.2.3 — OpenTelemetry

- **Что нужно знать:** трейсинг, метрики, логи (стандарт)
- **Вопросы на собеседовании:** "Как добавить трейсинг в [ASP.NET](#) Core?"
- **Практика:** Добавить OpenTelemetry в API и отправить в Jaeger

№ 4.4.2.4 — MassTransit

- **Что нужно знать:** обмен сообщениями (RabbitMQ, Kafka)
- **Вопросы на собеседовании:** "Как работать с очередями в [ASP.NET](#) Core?"
- **Практика:** Настроить MassTransit с RabbitMQ для отправки сообщений

№ 4.4.2.5 — SignalR

- **Что нужно знать:** WebSocket для реального времени
- **Вопросы на собеседовании:** "Как реализовать чат в реальном времени?"
- **Практика:** Создать SignalR-приложение для чата

№ 4.4.2.6 — Entity Framework Core

- **Что нужно знать:** ORM, миграции, LINQ-запросы
- **Вопросы на собеседовании:** "В чем преимущество EF Core перед [ADO.NET](#)?"
- **Практика:** Создать API с EF Core для работы с БД

№ 4.4.2.7 — Dapper

- **Что нужно знать:** легкий ORM для производительности
- **Вопросы на собеседовании:** "Когда использовать Dapper вместо EF Core?"
- **Практика:** Написать запросы с Dapper для high-load сценариев

№ 4.4.2.8 — Пул соединений с БД

- **Что нужно знать:** настройка, переиспользование
 - **Вопросы на собеседовании:** "Как работает пул соединений в .NET?"
 - **Практика:** Настроить MaxPoolSize для Npgsql
-

4.4.3. Практические задания (Middle → Senior Level)

№ 4.4.3.1

- **Задание:** Создать REST API для управления пользователями (CRUD) с аутентификацией
- **Что проверяет:** Знание [ASP.NET](#) Core

№ 4.4.3.2

- **Задание:** Добавить gRPC-эндпоинт и клиент на C++
- **Что проверяет:** Понимание gRPC

№ 4.4.3.3

- **Задание:** Реализовать middleware для обработки ошибок и логирования
- **Что проверяет:** Понимание конвейера

№ 4.4.3.4

- **Задание:** Настроить Polly для устойчивости к сбоям
 - **Что проверяет:** Знание устойчивости
-

РАЗДЕЛ 4.5: AVALONIA UI (КРОССПЛАТФОРМЕННЫЙ GUI) — SENIOR LEVEL

4.5.1. Avalonia (Senior Level)

№ 4.5.1.1 — Avalonia

- **Что нужно знать:** кроссплатформенный UI-фреймворк (Windows, Linux, macOS, WebAssembly)
- **Вопросы на собеседовании:** "Что такое Avalonia и чем он отличается от WPF?"
- **Практика:** Установить Avalonia и создать приложение

№ 4.5.1.2 — XAML

- **Что нужно знать:** разметка интерфейса, привязка данных (Binding)

- **Вопросы на собеседовании:** "Как работает Binding в XAML?"
- **Практика:** Создать форму с TextBox, привязанным к свойству ViewModel

№ 4.5.1.3 — MVVM Pattern

- **Что нужно знать:** Model-View-ViewModel, INotifyPropertyChanged
- **Вопросы на собеседовании:** "Что такое MVVM и зачем он нужен?"
- **Практика:** Реализовать MVVM для приложения с кнопкой, меняющей текст

№ 4.5.1.4 — Стилизация

- **Что нужно знать:** Styles, Themes, Resources, StyleInclude
- **Вопросы на собеседовании:** "Как задать общий стиль для всех кнопок?"
- **Практика:** Создать тему с общими стилями

№ 4.5.1.5 — Кастомные контролы

- **Что нужно знать:** наследование от UserControl, переопределение шаблонов
- **Вопросы на собеседовании:** "Как создать кастомный контрол в Avalonia?"
- **Практика:** Создать NumberBox с кнопками + и -

№ 4.5.1.6 — DataGrid

- **Что нужно знать:** отображение табличных данных, сортировка, фильтрация
- **Вопросы на собеседовании:** "Как отобразить коллекцию в DataGrid?"
- **Практика:** Привязать список пользователей к DataGrid

№ 4.5.1.7 — Навигация

- **Что нужно знать:** переход между окнами, DI для ViewModels
- **Вопросы на собеседовании:** "Как реализовать навигацию в Avalonia?"
- **Практика:** Создать приложение с несколькими окнами и навигацией

№ 4.5.1.8 — Асинхронность в UI

- **Что нужно знать:** async/await, Dispatcher

- **Вопросы на собеседовании:** "Как обновить UI из другого потока?"
- **Практика:** Загрузить данные асинхронно и обновить UI

№ 4.5.1.9 — P/Invoke в Avalonia

- **Что нужно знать:** вызов нативного кода
- **Вопросы на собеседовании:** "Как работать с нативным кодом в Avalonia?"
- **Практика:** Вызвать Windows API из Avalonia приложения

№ 4.5.1.10 — Сборка для разных платформ

- **Что нужно знать:** Desktop, Browser (WebAssembly), Mobile
- **Вопросы на собеседовании:** "Как собрать Avalonia приложение для WebAssembly?"
- **Практика:** Собрать приложение для WebAssembly и запустить в браузере

№ 4.5.1.11 — Плагины и модульность

- **Что нужно знать:** MEF, динамическая загрузка UI
- **Вопросы на собеседовании:** "Как сделать плагинную архитектуру в Avalonia?"
- **Практика:** Загрузить сборку с плагином и отобразить его UI

4.5.2. Практические задания (Senior Level)

№ 4.5.2.1

- **Задание:** Создать приложение на Avalonia с MVVM и навигацией между страницами
- **Что проверяет:** Знание Avalonia и MVVM

№ 4.5.2.2

- **Задание:** Создать DataGrid с редактированием и сохранением изменений
- **Что проверяет:** Работа с данными

№ 4.5.2.3

- **Задание:** Создать кастомный контрол с стилизацией и привязкой данных

- **Что проверяет:** Знание XAML и стилей

№ 4.5.2.4

- **Задание:** Собрать приложение для WebAssembly и запустить в браузере
 - **Что проверяет:** Кроссплатформенная сборка
-

🔗 СВОДНЫЙ ЧЕК-ЛИСТ ДЛЯ ПОДГОТОВКИ К СОБЕСЕДОВАНИЮ

Junior Level (должен знать 100%)

Базовый синтаксис: типы (value vs reference), циклы, условия

ООП: классы, наследование, полиморфизм, интерфейсы, абстрактные классы

Коллекции: List, Dictionary, HashSet, Queue, Stack

LINQ: Select, Where, GroupBy, OrderBy, Join

Исключения: try/catch/finally

Делегаты и события

Асинхронность: async/await базовое

[ASP.NET](#) Core: создание простого API

Middle Level (должен знать 100%)

Boxing/Unboxing, производительность

Асинхронность: Task, ValueTask, CancellationToken, WhenAll/WhenAny

Многопоточность: Thread, ThreadPool, Task, lock, Interlocked

Concurrent коллекции: ConcurrentDictionary, ConcurrentQueue

LINQ: отложенное выполнение, IQueryable, GroupJoin

GC: поколения, LOH, IDisposable

[ASP.NET](#) Core: DI (Scoped/Transient/Singleton), Middleware

Avalonia: XAML, Binding, MVVM

Senior Level (должен знать 100%)

CLR: GC (поколения, LOH), финализаторы, WeakReference

JIT: Tiered Compilation, Native AOT

Асинхронность: State Machine, ConfigureAwait(false), IAsyncEnumerable

Многопоточность: SemaphoreSlim, Barrier, SpinLock, ReaderWriterLockSlim

P/Invoke, маршalling, GCHandle

Span<T>, Memory<T>, ref struct

[ASP.NET](#) Core: Health Checks, Polly, OpenTelemetry, MassTransit

Avalonia: кастомные контролы, стилизация, P/Invoke

Staff Level (должен знать 100%)

Внутреннее устройство JIT: как компилирует IL

Native AOT: как работает, ограничения

System.Text.Json: внутреннее устройство

Рефлексия и генерация IL (System.Reflection.Emit)

Разработка плагинной архитектуры

Оптимизация производительности: профилирование с dotnet-counters

🔨 ПРАКТИЧЕСКИЙ ПРОЕКТ ДЛЯ ЗАКРЕПЛЕНИЯ ДОМЕНА 4

Задание: Реализовать систему для управления задачами (Task Management System) с использованием всего стека .NET.

Шаги:

1. **Бэкенд:** [ASP.NET](#) Core API с JWT-аутентификацией
2. **БД:** PostgreSQL с Entity Framework Core
3. **Очереди:** MassTransit + RabbitMQ для обработки задач
4. **Кеш:** Redis для кеширования
5. **Логи:** Serilog с записью в файл и OpenSearch
6. **Метрики:** Prometheus + Grafana
7. **Трейсы:** OpenTelemetry + Jaeger
8. **Фронтенд:** Avalonia UI Desktop приложение (с MVVM)
9. **Тесты:** xUnit + Moq для unit-тестов

Проверка:

- API должно выдерживать 1000 запросов/сек
- UI должно обновляться в реальном времени (SignalR)
- Кеш должен уменьшать время ответа в 10 раз